

Historia de usuario M6.1S3

Panel Administrativo Visual de "Eventify" - Interfaz y Modelo

Objetivo de la Historia de usuario

Desarrollar una interfaz web interactiva para los administradores del sistema utilizando Thymeleaf, integrando el patrón Model-View-Controller (MVC) para la gestión visual del catálogo y asegurando una navegación fluida mediante el uso de modelos y redirecciones.

Contexto de Negocio

Hasta ahora, la interacción con "Eventify" ha sido técnica (vía API). El negocio requiere que los coordinadores de eventos tengan un panel visual para ver la oferta, registrar nuevos lugares y gestionar el inventario de forma intuitiva desde el navegador. Es crucial que la interfaz sea dinámica: si no hay eventos, el sistema debe informar al usuario; si se guarda un registro, debe confirmar la acción visualmente.

Detalle de la HU

- **Como:** Coordinador de eventos
- **Quiero:** Acceder a un panel administrativo visual
- **Para:** Gestionar el catálogo de eventos de forma ágil y con una experiencia de usuario amigable.

TASK 1

Implementación del Motor de Plantillas (Thymeleaf)

1. **Vistas Dinámicas:** Crear archivos HTML en `src/main/resources/templates` utilizando directivas de Thymeleaf:
 - `th:each`: Para iterar y mostrar la tabla de eventos.
 - `th:text`: Para renderizar datos dinámicos del modelo.
 - `th:if` / `th:unless`: Para manejar estados condicionales (ej. mostrar mensaje de "Catálogo vacío").
2. **Formularios Web:** Crear formularios para el registro de `Event` y `Venue` utilizando `th:action` y `th:object`.

TASK 2

Controladores de Vista y Gestión del Modelo

1. **Controladores de Vista:** Implementar controladores (usando `@Controller`) que retornen nombres de plantillas HTML.
2. **El Objeto Model:** Utilizar `Model` para pasar listas y objetos desde el backend hacia la vista.

3. **Captura de Datos:** Usar la anotación `@ModelAttribute` para recibir los datos de los formularios en los métodos `POST`.
4. **Redirecciones:** Implementar `redirect:/ruta` tras operaciones exitosas para evitar el reenvío de formularios al refrescar la página (Patrón Post-Redirect-Get).

TASK 3

Documentación de Rutas y Calidad de Interfaz

1. **Separación de Rutas:** Configurar la aplicación para distinguir entre rutas de API (`/api/`) y rutas de interfaz web (`/admin/`).
2. **Swagger UI:** Asegurar que Swagger solo documente los endpoints de la API, manteniendo la interfaz de usuario limpia y separada en la documentación.
3. **Testing de Vistas:** Implementar pruebas con MockMvc para verificar que las rutas de la interfaz retornan el estado 200 OK, el nombre de la vista correcto y contienen los atributos esperados en el modelo.

Criterios de aceptación

Escenario 1: Visualización Centralizada (Camino Feliz)

- Dado que existen eventos registrados en el sistema.
- Cuando el coordinador accede al panel administrativo.
- Entonces el sistema debe renderizar una tabla HTML con la lista completa de eventos y lugares, mostrando sus datos clave.

Escenario 2: Gestión de Inventario Vacío (Caso de Borde)

- Dado que no hay registros en la base de datos.
- Cuando se consulta la vista de listado.
- Entonces el sistema no debe mostrar una tabla vacía, sino un mensaje amigable: "Actualmente no hay eventos programados".

Escenario 3: Registro y Redirección Exitosa

- Dado un formulario de nuevo evento completado.
- Cuando el usuario hace clic en "Guardar".
- Entonces el sistema debe procesar el dato, guardarlo en la BD y realizar una redirección automática al listado principal para mostrar el cambio.

Escenario 4: Validación de Navegación (Testing)

- Dado un test de integración de interfaz.
- Cuando se invoca la ruta del panel.

- Entonces el test debe confirmar que el objeto `Model`` contiene la lista de eventos necesaria para la vista.